

AQI Tower: A Simulation-First Digital Twin Prototype for Designing a Multi-Stage Air Purification Tower

[Student Name 1]
Dept. of Computer Science and
Engineering
[College Name]
[City], India
[email address]

[Student Name 2]
Dept. of Computer Science and
Engineering
[College Name]
[City], India
[email address]

[Student Name 3]
Dept. of Computer Science and
Engineering
[College Name]
[City], India
[email address]

[Student Name 4]
Dept. of Computer Science and
Engineering
[College Name]
[City], India
[email address]

[Project Guide Name]
Dept. of Computer Science and
Engineering
[College Name]
[City], India
[email address]

Abstract—Air purifier hardware is slow and costly to iterate, and a student team without laboratory access cannot afford many wrong builds. We therefore designed a tower-type purifier as a software system first. This paper describes AQI Tower, a version-controlled pipeline that links parametric CAD (FreeCAD scripted in Python), steady RANS airflow simulation (OpenFOAM), Python analysis tools and a browser-based Three.js viewer. In the sense used by Grieves, the result is a digital twin prototype: the virtual tower exists before the physical one, and the measurement tools that will later feed it are already written and tested. The architecture has four layers: geometry, simulation, visualization, and data and decision support. Every number shown to a user carries its source. The pipeline has already changed the design. A geometry revision cut the modelled pressure loss of the empty tower from 17.3 Pa to 3.5 Pa. The simulation exposed a 38–48% air bypass around the HEPA filter, which a sealing change removed; the sealed model predicts 1332 m³/h with 111.2 Pa across the filter. The same tools led us to reject a proposed water-spray stage and to choose a smaller 12 V indoor demonstrator as the first physical build. No physical measurement has been taken yet, and the paper states where each number comes from.

Keywords—digital twin, computational fluid dynamics, OpenFOAM, Three.js, parametric CAD, air purification, research software

I. INTRODUCTION

Fine particulate matter (PM_{2.5}) is one of the main environmental health risks in Indian cities, and the 2021 WHO guideline lowered the recommended annual PM_{2.5} level to 5 µg/m³ [1]. Portable purifiers help indoors, but what they deliver depends on airflow and filter efficiency together [2], [3]. Our final-year project began with a larger idea: a vertical tower that pushes a large volume of air through several filter stages.

We are computer science students. We had no fluid mechanics laboratory, no workshop and a modest budget. A wrong sheet-metal build would cost weeks and money we did not have. So we treated the tower the way we would treat a software product. The design is written as code and simulated

before anything is built, every intermediate file is kept under Git, and the tools that produce numbers are tested.

This paper reports that approach. Its contributions are:

- a four-layer architecture that connects parametric geometry, CFD, web visualization and data analysis through explicit file-level dependencies;
- a CFD setup in which the fan catalogue curve is compiled into an OpenFOAM boundary condition from a JSON file, so the operating airflow comes out of the simulation instead of being assumed;
- a Three.js viewer that places exported simulation fields next to labelled schematic geometry, so a reader can tell simulated data from illustration;
- unit-tested measurement and analysis tools that are ready for the first physical trial;
- design decisions made with the pipeline, including one where it caught a fault in our own model.

Section II places the work in the digital twin literature. Section III describes the architecture and Section IV the implementation rules we followed. Section V presents results and Section VI the limits.

II. BACKGROUND AND RELATED WORK

A. Digital Twins

Grieves and Vickers separate a digital twin prototype, which describes a product before it is built, from a digital twin instance tied to one physical unit [4]. Kritzinger et al. classify systems by how data moves. A digital model has no automatic data exchange with the physical object. A digital shadow receives data from it automatically, and a full digital twin exchanges data in both directions [5]. Tao et al. survey industrial applications and find that many reported twins are models in this sense [6]. By these definitions AQI Tower is today a digital model and a digital twin prototype, and we use the term with that qualification. The data layer in Section III-

D is written so that the system can become a digital shadow once sensors are connected.

B. Simulation in Air Cleaner Design

OpenFOAM is an open-source finite-volume library written in object-oriented C++ [7]. We use its standard $k-\epsilon$ turbulence model [8]. Particles between about $0.1 \mu\text{m}$ and $1 \mu\text{m}$ are poorly collected by both impaction and diffusion [9], which is why HEPA media are classified at their most penetrating particle size under EN 1822 [10]. Low-cost filter boxes built from PC fans and furnace filters, such as the open-source Tempest Brisa [11], show that simple designs can be useful indoors. The demonstrator in Section V-D follows that line.

III. SYSTEM ARCHITECTURE

The software is split into four layers (Fig. 1). Each layer has its own folder and tools and can be owned by one team member. Layers talk only through files: JSON, OpenFOAM dictionaries, CSV and exported field slices. One member can rerun a simulation without touching the viewer code, and another can change the viewer without rerunning CFD. Table I lists the technology stack.

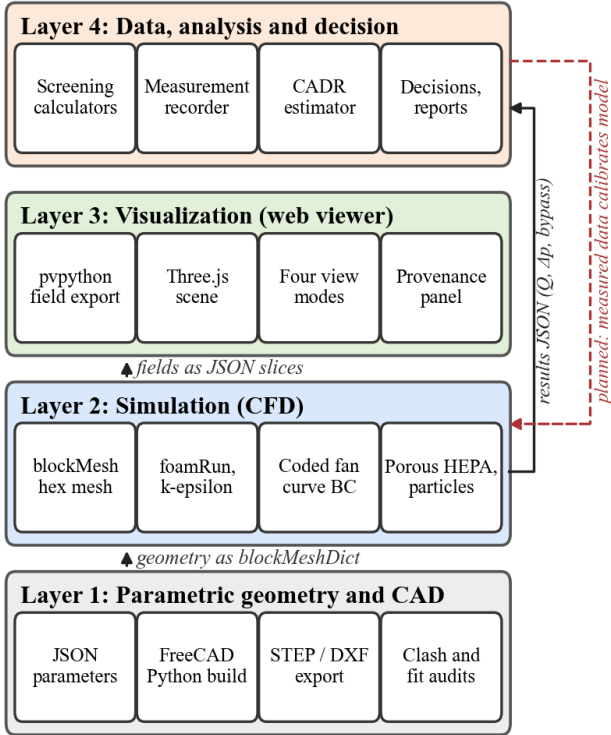


Fig. 1. Four-layer architecture of AQI Tower. Solid arrows show data flow implemented today; the dashed arrow is the measurement feedback still to be built.

TABLE I. TECHNOLOGY STACK BY LAYER

Layer	Tools and versions	Main outputs
1. Geometry	Python 3, FreeCAD 1.1.3 [14]	FCStd, STEP, DXF, audit JSON
2. Simulation	OpenFOAM 14 in WSL Ubuntu, blockMesh, foamRun	Fields; Q, Δp and bypass JSON
3. Visualization	ParaView 5.11 pvpython [12], Three.js r167 [13], Vite 5	JSON slices, 518 kB web build
4. Data and decision	Python standard library, NumPy, SciPy, Matplotlib, unittest	Checked CSV, CADR JSON, PDF reports
Shared	Git, Windows 11 with WSL, Node.js 24	Versioned history

A. Layer 1: Parametric Geometry and CAD

Tower dimensions are stored in JSON files under *cad/parametric*. Python scripts run inside FreeCAD [14], build solid models from these files, and export native FreeCAD and STEP files. For the indoor demonstrator they also write 1:1 DXF panel outlines. Changing a dimension means editing JSON and rerunning a script, not redrawing by hand. Each build runs a fit audit that checks pairs of solids for overlapping volume. The demonstrator assembly (73 objects) passed 435 panel and filter pair checks and 1,280 fastener checks with no clashes, and the current mechanical revision, with 493 objects, passed 119,805 pair checks. These audits find interference between nominal shapes. They say nothing about tolerances, strength or sealing.

B. Layer 2: Simulation

The CFD path avoids STL conversion. The tower is built from axis-aligned boxes, so a preparation script splits it into conformal rectangular blocks and writes a *blockMeshDict* directly. *blockMesh* then produces a hexahedral mesh with a 20 mm maximum edge, 63,780 cells for the GEO C geometry. OpenFOAM 14 runs in Ubuntu under WSL, using the *foamRun* solver in incompressible mode with steady RANS and the standard $k-\epsilon$ model with wall functions.

From a software point of view the fan model is the most interesting part. Instead of fixing an inlet velocity, a script reads the six-point catalogue curve of the Systemair KVO 250 fan [15] from JSON, logs the file's SHA-256 hash and generates a coded boundary condition that OpenFOAM compiles at run time. At each outer iteration the outlet suction pressure is set from the curve at the current flow rate, with a relaxation factor of 0.1. The airflow is therefore an output of the run.

The H13 HEPA filter is modelled as a porous baffle. The pressure jump across a baffle of depth L at face velocity v is

$$\Delta p = (D \mu v + \frac{1}{2} I \rho v^2) L \quad (1)$$

where D and I are the Darcy and inertial coefficients. The supplier publishes a single rated point (300 Pa at 2.84 m/s), so we set $I = 0$, fit D to that point, and bracket it with low, medium and high cases. A separate Python module tracks 224 particles of four sizes through the saved velocity field, using one-way coupling, gravity and first-order Euler steps.

C. Layer 3: Visualization

Simulation results usually stay inside ParaView on one laptop. We wanted the team, our guide and readers with no CFD background to open the model in a browser. A pvpython script [12] cuts scalar planes from the saved OpenFOAM fields and writes them as JSON. The viewer, written with

Three.js [13] and bundled by Vite into a 518 kB build, loads these slices as colour-mapped meshes beside the tower geometry (Fig. 2). It has four modes: ARCHITECTURE, PARTICLE, CFD SUMMARY and CFD FIELD.

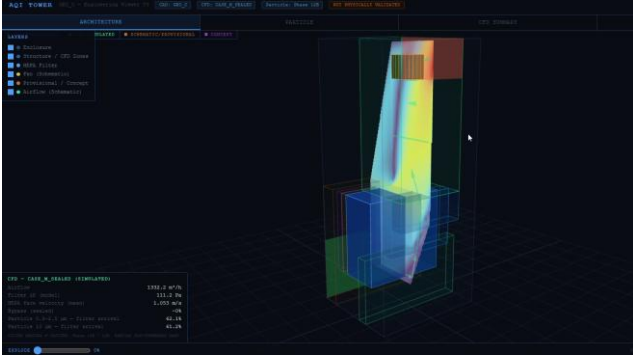


Fig. 2. Three.js viewer in ARCHITECTURE mode: an exported CFD velocity slice over the GEO_C tower, with the CASE_M_SEALED summary panel at lower left.

Every object in the scene is tagged. CFD slices are marked as simulated, geometry generated from configuration files is marked schematic, and ideas are marked as concepts. A provenance panel names the case and file behind each displayed number, and a banner states that nothing has been physically validated. This matters because a smooth colour plot looks convincing whether or not it came from a solver.

D. Layer 4: Data, Analysis and Decision

This layer holds the tools that turn results into decisions and the tools that will take real measurements. Screening calculators find fan-system operating points, pressure budgets and duct velocities. Decision matrices score design options against weighted criteria, and report scripts build the PDF engineering packages.

For physical trials, *record_measurements.py* imports readings from a JSON Lines file into a checked CSV. It records instrument identity, units and calibration references, converts timestamps to UTC, and rejects missing values, unit mismatches, duplicates and timestamps that run backwards. Missing samples stay missing; they are never filled in. *indoor_decay.py* then estimates the clean air delivery rate (CADR) from paired off and on decay periods in a room of volume V :

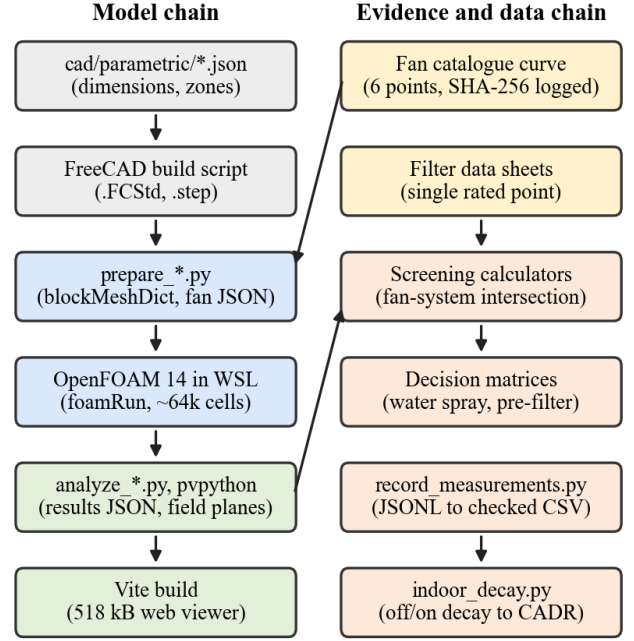
$$\text{CADR} = V(k_{\text{on}} - k_{\text{off}}) \quad (2)$$

where k is the negative slope of $\ln C(t)$ in each window. The script refuses to report a value unless the user explicitly declares the mixing and ventilation assumptions behind this model, and it never clips a negative result to zero. Both tools use only the Python standard library. They are covered by 44 unit tests that run on synthetic fixtures, since no real data exists yet.

E. Dependencies and Data Flow

Fig. 3 shows how files move between layers. The model chain on the left runs from JSON parameters through CAD, meshing, the solver, post-processing and the web build. The evidence chain on the right starts from external data such as the fan catalogue and filter data sheets, which feed both the simulation and the screening calculators. CFD results feed the calculators, and the calculators feed the decision matrices. The

measurement tools import no simulation code, so a measured value cannot be quietly mixed with a predicted one.



Colours match the layers in Fig. 1; yellow marks external source data.

Fig. 3. File-level dependencies. Left: model chain from parameters to the web viewer. Right: evidence and measurement chain.

Third-party dependencies are few. The viewer depends only on three and vite. The analysis scripts use NumPy, SciPy and Matplotlib, while the measurement tools use the standard library alone. FreeCAD and ParaView bring their own Python interpreters, so their scripts run inside those programs rather than in a shared environment.

IV. IMPLEMENTATION PRACTICES

Three rules shaped the code more than any library choice did.

- Every number carries a label. Documents and outputs mark values as FACT, DETECTED, ASSUMPTION, UNKNOWN or RECOMMENDATION, so a calculated operating point is never shown with the same status as a catalogue value.
- No invented data. The pressure-drop experiment has a data template and a fitting script, but every result field is null until a test is run, and the script handles an empty file without producing output.
- Outputs are reproducible and protected. Source files are hashed, scripts refuse to overwrite an existing output, and generated meshes and fields stay out of Git while the scripts that create them stay in.

The code base has about 12,600 lines of Python and JavaScript, including 66 Python scripts. All runs used one laptop (Intel Core i7-13650HX, 16 GB RAM); meshes of this size did not need a cluster.

V. RESULTS

A. Geometry Iteration

The first fan-driven model, V01, showed reverse flow at the turn into the clean-air riser, where air had to pass a 4.67:1

contraction followed at once by a 2.06:1 expansion. We tested three geometry changes. GEO_C combined a two-step turning chamber, a wider riser entry and removal of a blind cap above the outlet. As Table II shows, it cut the modelled total-pressure loss from 17.3 Pa to 3.5 Pa and reduced reverse flow in the riser. Because GEO_C changed three things at once, the gain cannot be attributed to any one of them.

TABLE II. EMPTY-TOWER CFD COMPARISON

Metric	V01	GEO_C
Operating flow (m ³ /h)	1475.7	1496.1
Total-pressure loss (Pa)	17.3	3.5
Peak cell speed (m/s)	6.53	5.62
Riser reverse-flow volume (%)	23.1	15.9
Riser-entry velocity CoV	1.087	0.234

^a Steady RANS with the KVO 250 curve, no filters. V01 was partially converged; GEO_C met the residual criteria. Simulated, not measured.

B. A Bypass Found in Our Own Model

Adding the HEPA filter raised the fan head 13 to 19 times while flow fell by only 4–6%, which looked encouraging. While extracting results we compared the flow through the filter face with the total flow, and they did not match. Between 38% and 48% of the air went around the filter through a path between the dirty plenum and the riser, and the share grew as filter resistance rose. A purifier that treats half its air cannot be rated at all. Adding frame baffles to close the path (case CASE_M_SEALED) brought the modelled bypass to about zero. The sealed case predicts 1332.2 m³/h, 115 Pa of fan head, 111.2 Pa across the filter and a face velocity of 1.053 m/s. A real housing still has to be leak-tested; the numerical seal only shows that the model is closed.

Particle tracking in the sealed field found that 62.1% of the 0.3, 1 and 2.5 μ m particles and 61.2% of the 10 μ m particles reach the filter face. The rest land on walls and none leave through the outlet. Reaching the filter is not the same as being captured. We have no size-resolved efficiency curve for the chosen filter, so no CADR figure follows from this result.

C. Pressure Budget and Stage Decisions

Fig. 4 overlays the fan curve with system curves built from the sealed result. A clean G4 pre-filter of about 25 Pa, an estimate pending supplier data, moves the operating point from 1332 to about 1300 m³/h. A loaded pre-filter at 100 Pa moves it to about 1214 m³/h.

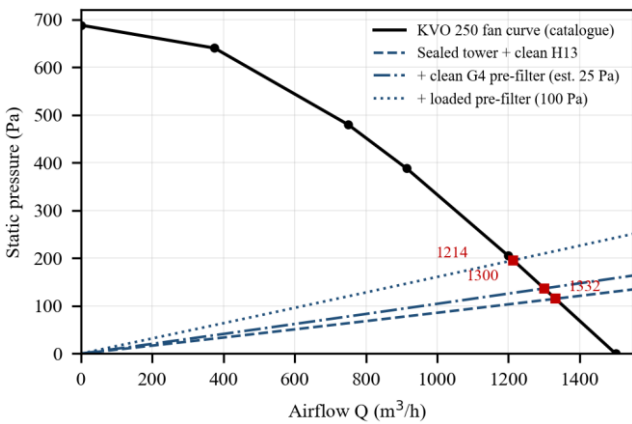


Fig. 4. Fan curve and analytical system curves for the sealed tower. Squares mark operating points. Pre-filter resistances are estimates, not measurements.

The same analysis was used on a proposed water-spray stage. Spray scrubbing does little for particles between 0.1 μ m

and 2 μ m [9], it adds 35–125 Pa, and it brings water hygiene risk into a device that blows air at people. In an eleven-criterion weighted matrix it scored 13 out of 100 against 84 for a dry G4/M5 pre-filter, and we dropped it.

The pipeline also corrected us. An earlier note claimed 57–67 Pa of spare fan head for a future carbon bed. A later screening script compared all losses at one common flow and showed that this margin does not exist at 1300 m³/h, where the clean HEPA and pre-filter already use nearly all of the fan curve. Because the claim came from a script and not from a slide, it was easy to recheck and withdraw.

D. From Full Tower to Indoor Demonstrator

The full tower needs a 301 W, 20 kg fan, sheet-metal fabrication and rated mains wiring, and each of these still has open review items. We therefore defined a smaller first build, D01 (Fig. 5). It is a 550 mm plywood cabinet with two opposed MERV13 filters and four 140 mm, 12 V PC fans. MERV13 is not HEPA, and D01 makes no claim about gases or outdoor air. Its job is to let us measure flow, filter pressure, power, noise and room decay with the Layer 4 tools. Screening with digitized fan curves and assumed filter resistance gives 309–377 m³/h. These are scenarios, not a predicted rating.

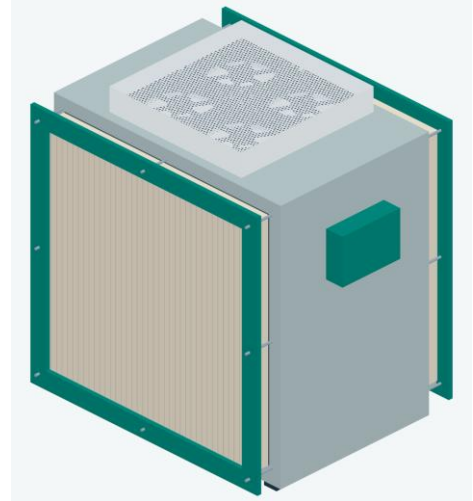


Fig. 5. CAD render of the D01 indoor demonstrator, generated by the Layer 1 scripts. Review model only; not released for fabrication.

VI. LIMITATIONS AND FUTURE WORK

The main limits are plain:

- No physical prototype exists. Every performance figure in this paper is simulated or calculated.
- Each RANS case uses one mesh resolution with no independence study, and the k - ϵ model is known to be weak in strongly separated flow [8]. Some cases did not meet residual targets, although integrated quantities were stable.
- The HEPA model rests on one supplier data point, and the pre-filter and carbon stages have no measured resistance.
- The viewer reads exported files and has no live data link, so the system is a digital model and not yet a digital shadow [5].

The next steps follow the dashed arrow in Fig. 1. We will build D01, collect off and on decay data and filter pressure with the existing recorder, fit porous resistance coefficients to the measured pressure drop, and feed those coefficients back into the CFD. A live acquisition adapter for the chosen sensors and a streaming panel in the viewer would then turn the model into a digital shadow.

VII. CONCLUSION

Treating a hardware design as a software project worked for a team without a laboratory. Writing geometry as data, running CFD from scripts and publishing results in a browser let us split the work by layer and check each other's numbers. Several of the most useful outputs were negative: a filter bypass, a stage we should not build and a pressure margin that did not exist. Each would have been expensive to discover after fabrication. The tools are now waiting on the one input software cannot supply, which is measured data from the first build.

ACKNOWLEDGMENT

We thank [Project Guide Name] for guidance and [funding body] for financial support. AI coding assistants (OpenAI ChatGPT/Codex and Anthropic Claude Code) were used to help write scripts and documentation in this project and to draft this paper; the authors are responsible for checking all content and results.

REFERENCES

- [1] World Health Organization, *WHO Global Air Quality Guidelines: Particulate Matter (PM_{2.5} and PM₁₀), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva, Switzerland: WHO, 2021.
- [2] R. J. Shaughnessy and R. G. Sextro, "What is an effective portable air cleaning device? A review," *J. Occup. Environ. Hyg.*, vol. 3, no. 4, pp. 169–181, 2006.
- [3] U.S. Environmental Protection Agency, "Guide to air cleaners in the home," 2nd ed., EPA 402-F-08-004, 2018.
- [4] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.
- [5] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [6] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.
- [7] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Comput. Phys.*, vol. 12, no. 6, pp. 620–631, 1998.
- [8] B. E. Launder and D. B. Spalding, "The numerical computation of turbulent flows," *Comput. Methods Appl. Mech. Eng.*, vol. 3, no. 2, pp. 269–289, 1974.
- [9] W. C. Hinds, *Aerosol Technology: Properties, Behavior, and Measurement of Airborne Particles*, 2nd ed. New York, NY, USA: Wiley, 1999.
- [10] *High Efficiency Air Filters (EPA, HEPA and ULPA) – Part 1: Classification, Performance Testing, Marking*, EN 1822-1:2019, CEN, 2019.
- [11] Tempest Brisa open-source air purifier, GitHub repository. [Online]. Available: <https://github.com/obife29/Tempest-Brisa-Open-Source>
- [12] J. Ahrens, B. Geveci, and C. Law, "ParaView: An end-user tool for large data visualization," in *The Visualization Handbook*, C. D. Hansen and C. R. Johnson, Eds. Burlington, MA, USA: Elsevier, 2005, pp. 717–731.
- [13] three.js JavaScript 3D library, release r167. [Online]. Available: <https://threejs.org>
- [14] FreeCAD, open-source parametric 3D modeler, version 1.1. [Online]. Available: <https://www.freecad.org>
- [15] Systemair, "KVO rectangular duct fans," technical data sheet. [Online]. Available: <https://www.systemair.com>